Numerical Approximation of the Spectrum for the Dirichlet Laplacian

Ebube Anozie

April 24, 2026

Department of Applied Mathematics
University of Colorado Boulder

APPM 6470 – Prof. Nancy Rodriguez

Abstract

The Dirichlet Laplacian is a fundamental elliptic operator whose eigenvalues describe physical quantities such as the resonant frequencies of a drumhead or the energy levels of a quantum particle confined to a box [**balakrishnan2022particle**]. In dimensions $n \geq 2$, it is generally not possible to compute the spectrum explicitly, apart from a few exceptions such as the ball or a product of intervals. For most domains, one must rely on numerical approximation (e.g., finite elements). In this report, we study how to approximate the Laplacian using the finite element method. We introduce Sobolev spaces, explain why compactness guarantees a discrete spectrum, discretize the problem using linear finite elements, and numerically verify the optimal $O(h^2)$ convergence rate for eigenvalues on the unit square. The results confirm classical error estimates and lay the groundwork for adaptive methods on non-convex domains.

Contents

List of Figures

1 Introduction and Motivation

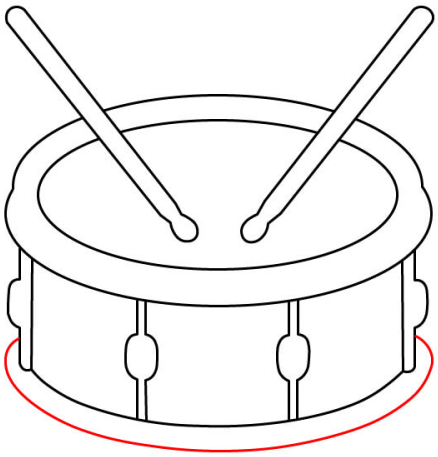


Figure 1: A drum

When we see a drum, we have an idea of how it would sound. Mathematically, with geometric information about a domain, we hope to obtain spectral information. For a few simple shapes, the spectrum is known exactly. But for most domains with irregular shapes, or holes, exact formulas for eigenvalues do not exist.

The spectrum of differential operators has very important physical interpretations: from the modulational stability problem for traveling periodic waves in an infinitely deep fluid to solving models for the quantum-mechanical system of the hydrogen atom. We use it in the study of PDE models across a wide range of areas, including acoustic waves, mechanical oscillations, aeronautics, and quantum physics.

For instance, the Laplace operator with Dirichlet boundary conditions[1] is the partial differential equation

$$\begin{aligned} -\Delta u &= \lambda u \quad \text{in } \Omega, \\ u &= 0 \quad \text{on } \partial\Omega, \end{aligned} \tag{1}$$

This problem appears throughout physics where the eigenvalues represent the natural frequencies of a vibrating membrane, and the energy levels of a quantum particle confined to a box [**girouard2017geometric**].

In this report, we will show:

1. How to approximate an infinite-dimensional eigenvalue problem using the Galerkin + finite element method (FEM).

[1] the number λ is called an eigenvalue and the corresponding function u the eigenfunction.

2. How compact embedding implies discrete spectrum.

3. That FEM attains optimal convergence.

4. How degeneracies, nodal lines, and orthogonality all emerge naturally from the discretization, confirming that FEM preserves the essential spectral structure.

2 Mathematical Background: Sobolev Spaces and the Dirichlet Laplacian

The eigenvalue problem 1 is naturally set in the framework of Sobolev spaces. The definitions and theorems we give below follow standard references such as [**evans2022partial**], [**brezis2011functional**], [**hunter2014notes**], [**trefethen2022numerical**], and [**salsa2016partial**].

Definition 2.1 ($L^2(\Omega)$ space)**.** *$L^2(\Omega)$ is the space of Lebesgue measurable functions $u : \Omega \to \mathbb{R}$ such that*

$$\int_\Omega |u(x)|^2 \, dx < \infty.$$

This space is a Hilbert space with inner product

$$\langle u, v \rangle_{L^2} = \int_\Omega uv \, dx$$

and induced norm

$$\|u\|_{L^2} = \sqrt{\langle u, u \rangle}.$$

Definition 2.2 (Sobolev space $H^1(\Omega)$)**.** *The Sobolev space $H^1(\Omega)$ consists of all functions $u \in L^2(\Omega)$ whose first partial derivatives $\frac{\partial u}{\partial x_1}, \frac{\partial u}{\partial x_2}$ (understood in the weak sense) also belong to $L^2(\Omega)$. The H^1 norm is*

$$\|u\|_{H^1}^2 = \|u\|_{L^2}^2 + \|\nabla u\|_{L^2}^2,$$

where $\nabla u = (\frac{\partial u}{\partial x_1}, \frac{\partial u}{\partial x_2})$.

Definition 2.3 ($H_0^1(\Omega)$)**.** *The space $H_0^1(\Omega)$ is the closure of $C_c^\infty(\Omega)$ (infinitely differentiable functions with compact support in Ω) with respect to the H^1 norm. Intuitively, we can interpret $H_0^1(\Omega)$ to consist of functions in $H^1(\Omega)$ that vanish on the boundary $\partial\Omega$ in the sense of traces.*

Definition 2.4 (Coercive)**.** *A bilinear form $a(\cdot, \cdot)$ on a Hilbert space V is called coercive if there exists a constant $\alpha > 0$ such that*

$$a(v, v) \geq \alpha \|v\|_V^2 \quad \text{for all } v \in V.$$

For $a(u, v) = \int_\Omega \nabla u \cdot \nabla v$ on $V = H_0^1(\Omega)$, the Poincaré inequality $\|v\|_{L^2} \leq C\|\nabla v\|_{L^2}$ yields

$$a(v, v) = \|\nabla v\|_{L^2}^2 \geq \frac{1}{1 + C^2} \|v\|_{H^1}^2,$$

so $a(\cdot, \cdot)$ is coercive on $H_0^1(\Omega)$.

Definition 2.5 (Compact embedding)**.** *A Banach space X is compactly embedded into another Banach space Y if every bounded sequence in X has a subsequence that converges strongly in Y. We write $X \hookrightarrow Y$.*

Definition 2.6 (Approximability)**.** *The family $\{V_h\}$ is **approximating** if for every $u \in H_0^1(\Omega)$,*

$$\lim_{h\to 0} \inf_{v_h \in V_h} \|u - v_h\|_{H^1} = 0.$$

Theorem 2.1 (Riesz representation theorem)**.** *Let H be a Hilbert space with inner product $\langle \cdot, \cdot \rangle$. For every continuous linear functional $f : H \to \mathbb{R}$, there exists a unique $u_f \in H$ such that $f(v) = \langle u_f, v \rangle$ for all $v \in H$.*

In the next subsection, we derive the weak formulation of our eigenvalue problem.

2.1 Weak Formulation

To study the eigenvalue problem 1, we multiply by a test function $v \in H_0^1(\Omega)$ and integrate by parts. Integration by parts (Green's identity) gives:

$$\int_\Omega (-\Delta u) v \, dx = \int_\Omega \nabla u \cdot \nabla v \, dx - \int_{\partial\Omega} \frac{\partial u}{\partial n} v \, ds.$$

Since $v = 0$ on the boundary, the boundary term vanishes. Thus we obtain the **weak formulation**:
Find $(\lambda, u) \in \mathbb{R} \times H_0^1(\Omega)$, $u \neq 0$, such that

$$a(u, v) = \lambda \, b(u, v) \quad \forall v \in H_0^1(\Omega), \tag{2}$$

where

$$a(u, v) = \int_\Omega \nabla u \cdot \nabla v \, dx \quad \text{(bilinear form)}, \qquad b(u, v) = \int_\Omega uv \, dx \quad \text{(inner product in } L^2)$$

[**adams2003sobolev**]. The Riesz representation theorem 2.1 guarantees that for any continuous linear functional f on $H_0^1(\Omega)$, there exists a unique $u \in H_0^1(\Omega)$ such that $a(u, v) = f(v)$ for all $v \in H_0^1(\Omega)$.

One might ask whether the eigenfunctions belong to the more regular space $H^2(\Omega)$, since the Laplacian involves second derivatives. On smooth domains, elliptic regularity guarantees $u \in H^2(\Omega) \cap H_0^1(\Omega)$ for solutions of $-\Delta u = f$ with $f \in L^2(\Omega)$ but may fail on Lipschitz domains for the worst f. However, for the variational formulation and the convergence analysis of the finite element method, the space $H_0^1(\Omega)$ is sufficient [**grisvard2011elliptic**]. The eigenfunctions are smooth in the interior and the additional H^2 regularity, when available, only improves the convergence rate.

How can we turn an abstract, infinite-dimensional eigenvalue problem into something a computer can solve?

3 Finite Element Discretization

The discretization of the eigenvalue problem 2 through finite elements relies on two concepts [**girouard2017geometric**, **brenner2008mathematical**]:

(i) the Galerkin idea: projecting the problem onto a finite-dimensional subspace $V_h \subset H_0^1(\Omega)$;

(ii) the choice of the subspace V_h using polynomials on a mesh.

3.1 Galerkin Approximation

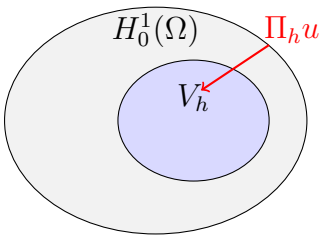


Figure 2: Galerkin projection

Let $V_h \subset H_0^1(\Omega)$ be a finite-dimensional subspace. We now want to find $(\lambda_h, u_h) \in \mathbb{R} \times V_h$, $u_h \neq 0$, such that

$$a(u_h, v_h) = \lambda_h \, b(u_h, v_h) \quad \forall v_h \in V_h. \tag{3}$$

If we choose a basis $\{\phi_j\}_{j=1}^N$ for V_h, then

$$u_h = \sum_{j=1}^{N} u_j \phi_j$$

. Taking $v_h = \phi_i$ for each i yields the **discretised matrix eigenvalue problem**:

$$A_h \mathbf{u} = \lambda_h M_h \mathbf{u}, \tag{4}$$

where

$$(A_h)_{ij} = a(\phi_j, \phi_i) \quad \text{(stiffness matrix)}^1, \qquad (M_h)_{ij} = b(\phi_j, \phi_i) \quad \text{(mass matrix)}^2$$

A_h is SPD due to coercivity, and M_h is SPD due to inner product positivity.

Remark 3.1. *If $\{V_h\}$ is approximating (dense in H_0^1 as $h \to 0$), then the finite-element eigenvalues converge to the exact eigenvalues.*

[1] the matrix of the discretized Laplacian.

[2] it generates the bilinear form of the L^2-inner product on the finite element space.

3.2 P_1 Finite Elements on Triangulations

Definition 3.1 (Triangulation)**.** *A **triangulation** $\mathcal{T}_h$ of Ω is a partition of Ω into a finite number of triangles such that any two triangles intersect at most at a common vertex or edge. The parameter $h = \max_{T\in\mathcal{T}_h} diam(T)$ is the **mesh size**. See Figure 3.*

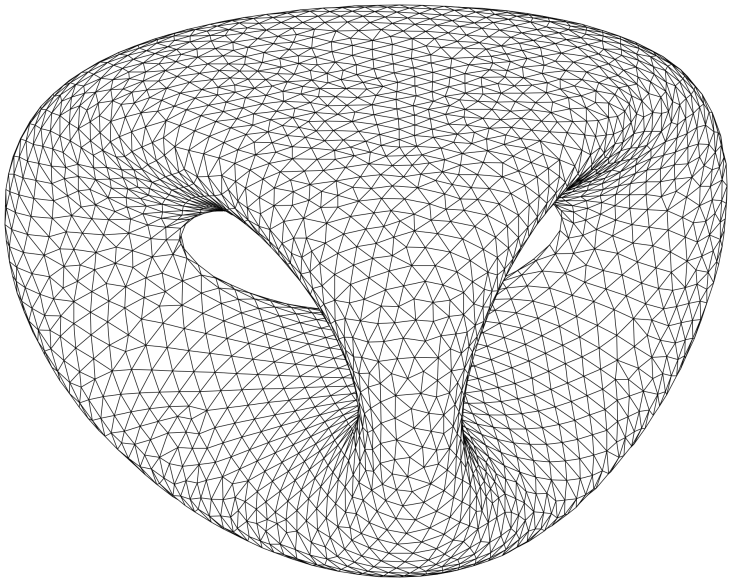


Figure 3: Example triangulation (adapted from Wikipedia)

Definition 3.2 (P_1 finite element space)**.** *The space V_h consists of all continuous functions that are linear (affine) on each triangle $T \in \mathcal{T}_h$ and vanish on the boundary $\partial\Omega$:*

$$V_h = \left\{ v_h \in C(\overline{\Omega}) : v_h|_T \in P_1(T)\ \forall T \in \mathcal{T}_h,\ v_h|_{\partial\Omega} = 0 \right\}.$$

Here $P_1(T)$ denotes the set of all linear functions $a + bx + cy$ on triangle T.

Since $C_c^\infty(\Omega)$ is dense in $H_0^1(\Omega)$ and the P_1 nodal interpolant $I_h u$ satisfies $\|u - I_h u\|_{H^1} \leq Ch\|u\|_{H^2}$ for $u \in H^2(\Omega)$, the family $\{V_h\}$ is indeed approximating 2.6.

For a triangle T with vertices x_i, x_j, x_k and area $|T|$, the element stiffness matrix entries are

$$A_{pq}^T = \int_T \nabla\phi_p \cdot \nabla\phi_q \, dx = \frac{1}{4|T|}(b_p \cdot b_q),$$

where b_p is the vector perpendicular to the edge opposite vertex p. The element mass matrix entries are

$$M_{pq}^T = \int_T \phi_p \phi_q \, dx = \frac{|T|}{12}(1 + \delta_{pq}),$$

where $\delta_{pq} = 1$ if $p = q$ and 0 otherwise. After assembling all element contributions, we get the sparse discretised eigenvalue problem (4) and solve with an iterative eigensolver (implemented in MATLAB as `eigs`).

In the following subsection, we show why compactness is important.

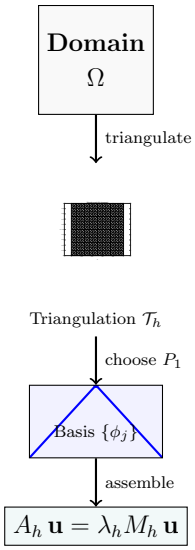


Figure 4: FEM pipeline

4 Compact Embedding and Discrete Spectrum

On unbounded domains like $\Omega = \mathbb{R}^n$, the Laplacian has purely continuous spectrum $[0, \infty)$ with no eigenvalues. Without compactness, continuous spectrum can appear. Compactness together with coercivity is therefore essential for the eigenvalue problem to be well-approximable by finite matrices. Before we give the proof of this important result, we state

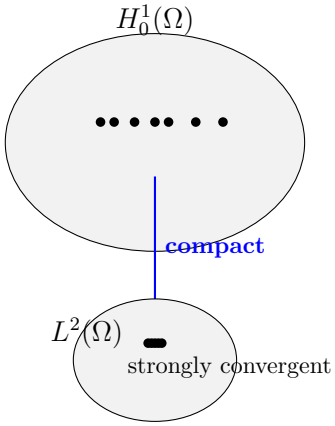


Figure 5: Compact embedding

a few lemmas.

Lemma 4.1 (Rellich-Kondrachov)**.** *For a bounded Lipschitz domain $\Omega \subset \mathbb{R}^2$, the embedding $H_0^1(\Omega) \hookrightarrow L^2(\Omega)$ is compact.*

Lemma 4.2 (Gårding's inequality)**.** *For all $u \in H_0^1(\Omega)$,*

$$a(u,u) \geq C_1 \|u\|_{H^1}^2 - \gamma \|u\|_{L^2}^2,$$

where $C_1 > 0$ and $\gamma \in \mathbb{R}$ are constants depending on L and Ω.

Theorem 4.3. *Let L be a symmetric, uniformly elliptic operator on a bounded domain $\Omega \subset \mathbb{R}^d$, with Dirichlet boundary conditions. Then L has an increasing sequence of real eigenvalues of finite multiplicity*

$$\lambda_1 \leq \lambda_2 \leq \lambda_3 \leq \cdots \leq \lambda_n \leq \cdots$$

such that $\lambda_n \to \infty$ as $n \to \infty$. Moreover, there exists an orthonormal basis $\{\phi_n : n \in \mathbb{N}\}$ of $L^2(\Omega)$ consisting of eigenfunctions $\phi_n \in H_0^1(\Omega)$ satisfying

$$L\phi_n = \lambda_n \phi_n.$$

Proof. We choose $\mu > 0$ large enough so that the shifted operator $L + \mu I$ is coercive on $H_0^1(\Omega)$. The inverse $K := (L + \mu I)^{-1}$ is then a well-defined, compact, self-adjoint operator on $L^2(\Omega)$; compactness follows from the Rellich–Kondrachov 4.1.

By the spectral theorem for compact self-adjoint operators, the non-zero spectrum of K consists of real eigenvalues $\{\kappa_n\}$ of finite multiplicity that accumulate only at 0. The corresponding eigenvectors form an orthonormal basis of $L^2(\Omega)$.

The eigenvalue relation $K\psi = \kappa\psi$ ($\kappa \neq 0$) is equivalent to $L\psi = \lambda\psi$ with $\lambda = \kappa^{-1} - \mu$. Since $\kappa_n \to 0$, we obtain $\lambda_n \to +\infty$. Gårding's inequality 4.2 provides the lower bound $\lambda \geq -\gamma$, so only finitely many eigenvalues are negative. After ordering, the spectrum of L is a discrete sequence

$$\lambda_1 \leq \lambda_2 \leq \lambda_3 \leq \cdots \leq \lambda_n \leq \cdots, \qquad \lambda_n \to \infty,$$

and the eigenfunctions $\{\psi_n\}$ form an orthonormal basis of $L^2(\Omega)$. □

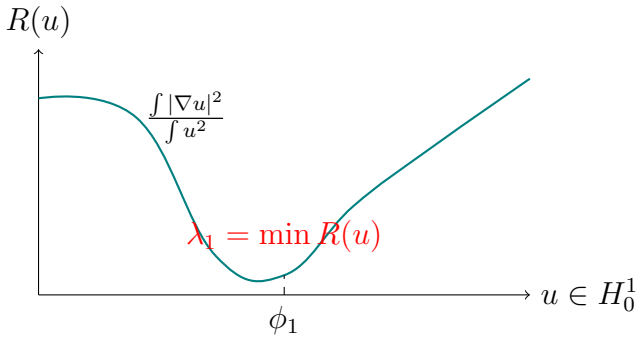
Remark 4.4. *Let us consider the solution operator $T : L^2(\Omega) \to L^2(\Omega)$ defined by $Tf = u$ where u solves $-\Delta u = f$ with $u|_{\partial\Omega} = 0$. Because $H_0^1(\Omega) \hookrightarrow L^2(\Omega)$ is compact, the operator T is compact. The eigenvalues of T are $\mu_k = 1/\lambda_k$, so $\lambda_k \to \infty$. This is why the Dirichlet Laplacian has a discrete spectrum: eigenvalues $\lambda_1 < \lambda_2 \leq \lambda_3 \leq \cdots \to \infty$, each of finite multiplicity.*

5 Principal eigenvalue and variational characterization

A central result in spectral theory is the **variational characterization** of the smallest eigenvalue[1]:

$$\lambda_1 = \inf_{u \in H_0^1(\Omega)\setminus\{0\}} \frac{\int_\Omega |\nabla u|^2 dx}{\int_\Omega u^2 \, dx}. \tag{5}$$

[1] λ_1 is also known as the principal eigenvalue.



This formulation involves minimization over an infinite-dimensional space. However, even simple geometries require advanced special functions. For the unit disk in $\mathbb{R}^2$, eigenvalues are $\lambda = j_{m,k}^2$ where $j_{m,k}$ is the k-th zero of the Bessel function J_m. In $\mathbb{R}^3$, spherical Bessel functions appear [**girouard2017geometric**]. For domains with irregular shapes, cracks, or heterogeneous materials, numerical methods become essential.

The ratio on the right-hand side of 5 is called the Rayleigh quotient for the nonzero function $u \in H_0^1(\Omega)$:

$$R(u) = \frac{a(u,u)}{b(u,u)} = \frac{\int_\Omega |\nabla u|^2 dx}{\int_\Omega u^2 dx}.$$

We state some important theorems below without proof:

Theorem 5.1 (Rayleigh-Ritz)**.** *The eigenvalues of the Dirichlet Laplacian satisfy*

$$\lambda_1 = \min_{u \in H_0^1(\Omega)\setminus\{0\}} R(u),$$

*and for $k \geq 2$ the Min-Max formula [***girouard2017geometric***]:*

$$\lambda_k = \min_{\substack{S \subset H_0^1(\Omega) \\ \dim S = k}} \max_{u \in S\setminus\{0\}} R(u).$$

A question one could ask is: **Why does the Rayleigh quotient characterize λ_1?**

To see this, we take the orthonormal eigenfunctions $\{\phi_k\}$ satisfying $-\Delta\phi_k = \lambda_k\phi_k$ and $\|\phi_k\|_{L^2} = 1$. Any $u \in H_0^1$ can be expanded as $u = \sum_{k=1}^\infty c_k\phi_k$:

$$a(u,u) = \sum_{k=1}^\infty \lambda_k |c_k|^2, \qquad b(u,u) = \sum_{k=1}^\infty |c_k|^2.$$

Therefore

$$R(u) = \frac{\sum_{k=1}^\infty \lambda_k |c_k|^2}{\sum_{k=1}^\infty |c_k|^2} \geq \lambda_1 \frac{\sum_{k=1}^\infty |c_k|^2}{\sum_{k=1}^\infty |c_k|^2} = \lambda_1,$$

with equality if and only if $c_k = 0$ for all $k \geq 2$, i.e., u is a multiple of ϕ_1. Hence $\lambda_1 = \min R(u)$.

6 Numerical Experiment

We now apply finite elements methods to the Dirichlet eigenvalue problem 1 on a unit square where the exact principal eigenvalue is $\lambda_1 = 2\pi^2 \approx 19.7392088021787$. We discretise based on a uniform triangulation to get the generalised eigenvalue problem:

$$A_h \mathbf{u} = \lambda_h M_h \mathbf{u},$$

where A_h is the stiffness matrix and M_h the mass matrix.

6.1 Mesh Generation

We first partition the unit square by first creating an $(N+1) \times (N+1)$ grid of points (see Figure 6).

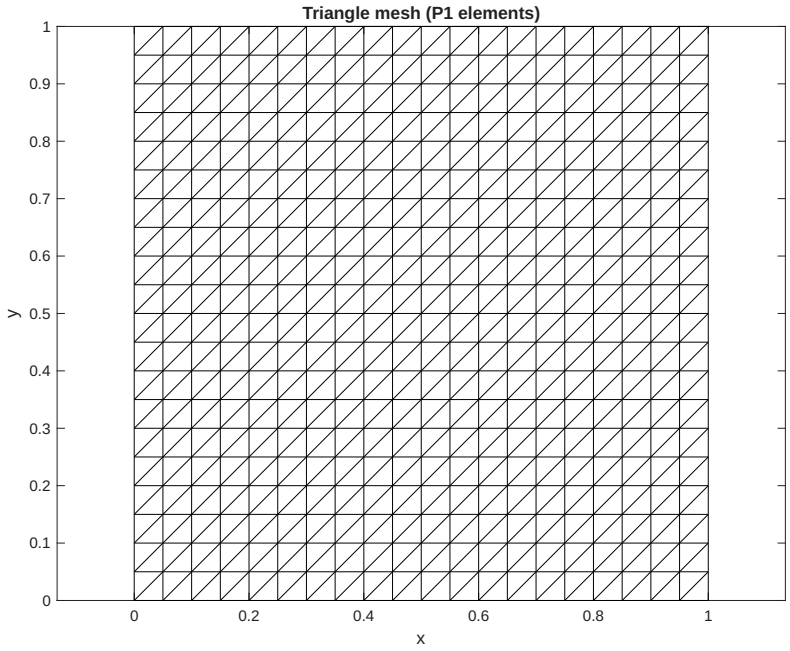


Figure 6: Each small square (size $h = 1/N$) is split into two right triangles by drawing the diagonal from bottom-left to top-right.

For a square with indices (i, j), the four corner nodes are:

```
bl = (j-1)*(N+1) + i;    % bottom-left
br = bl + 1;             % bottom-right
tl = j*(N+1) + i;        % top-left
tr = tl + 1;             % top-right
```

We stored the two triangles as:

```
elements(idx, :) = [bl, br, tr];    % first triangle
elements(idx+1, :) = [bl, tr, tl]; % second triangle
```

So the total number of elements is $2N^2$.

6.2 Assembly of Stiffness and Mass Matrices

For each triangle we compute the local matrices and add them to the global sparse matrices A and M.

Given the three vertices $(x_1, y_1), (x_2, y_2), (x_3, y_3)$, the area is

$$\text{area} = \frac{1}{2}\big|(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)\big|.$$

For linear triangular elements, the gradient of the basis function ϕ_i is constant:

$$\nabla\phi_i = (b_i, c_i)^T, \quad b_i = \frac{y_j - y_k}{2\,\text{area}}, \quad c_i = \frac{x_k - x_j}{2\,\text{area}},$$

where (i, j, k) is a cyclic permutation of $(1, 2, 3)$. Thus the (local) stiffness matrix:

$$K_{\text{local}}(i, j) = \text{area} \cdot (b_i b_j + c_i c_j).$$

and the (local) mass matrix:

$$M_{\text{local}} = \frac{\text{area}}{12}\begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

6.3 Imposing Dirichlet Boundary Conditions

Nodes on the boundary satisfy $x = 0$, $x = 1$, $y = 0$ or $y = 1$ (within a small tolerance). These degrees of freedom (DOF) are removed from the eigenvalue problem because the eigenfunction vanishes there. We create a logical vector `interior_nodes` that is `true` for interior nodes and `false` for boundary nodes. Then we extract the sub-matrices:

```
A_int = A(interior_nodes, interior_nodes);
M_int = M(interior_nodes, interior_nodes);
```

The number of interior DOFs is $(N - 1)^2$.

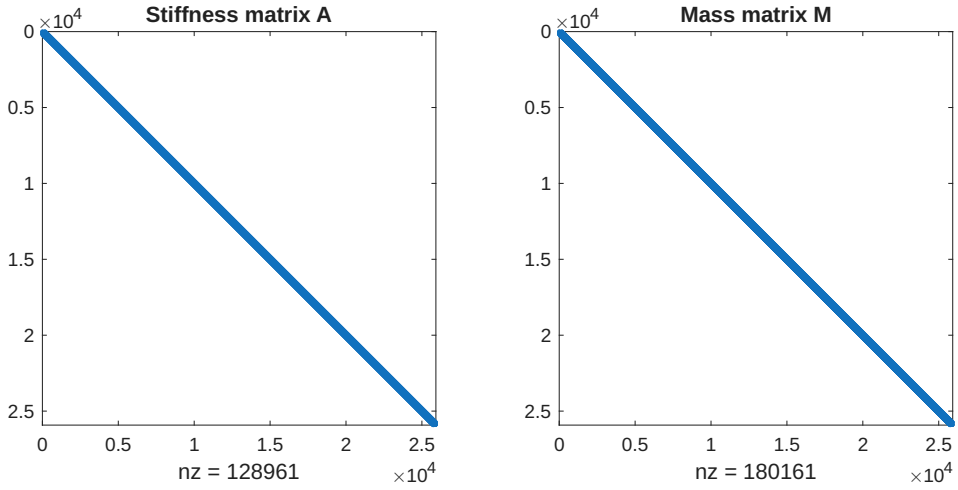


Figure 7: Sparsity pattern of the global stiffness matrix A (left) and mass matrix M (right) for the unit square with $N = 40$.

6.4 Results

In Figure 7 we show the sparsity patterns of the global stiffness matrix A and mass matrix M. Both matrices are extremely sparse.

For each mesh size $h = 1/N$, we compute:

- $\lambda_{1,h}$
- Error $e_h = |\lambda_{\text{exact}} - \lambda_{1,h}|$
- Ratio $r_h = e_h/e_{h/2}$ (expected to approach 4 for quadratic convergence)

Figure 8 verifies the optimal $O(h^2)$ convergence in theory. In Figure 9, we show the finite element approximation of the principal eigenfunction $\phi_{1,h}(x, y)$ for the Dirichlet Laplacian on the unit square. $\phi_{1,h}$ is positive in the interior of the square and vanishes on the boundary, as required by the Dirichlet condition. The single "bell-shaped" peak at the centre $(0.5, 0.5)$ matches the exact eigenfunction's ($\phi_1(x, y) = \sin(\pi x)\sin(\pi y)$) symmetry and lack of nodal lines (except the boundary). The smooth, symmetric shape observed numerically confirms that the finite element method successfully finds this minimiser.

Convergence of the principal eigenvalue $\lambda_{1,h}$ on the unit square.

h	Deg. of Freedom	$\lambda_{1,h}$	Error	Ratio
0.1000	121	20.2284	4.89e-01	---
0.0500	441	19.8611	1.22e-01	4.01
0.0250	1681	19.7697	3.04e-02	4.00
0.0125	6561	19.7468	7.61e-03	4.00
0.0063	25921	19.7411	1.90e-03	4.00

Figure 8: Convergence of the principal eigenvalue $\lambda_{1,h}$ on the unit square.

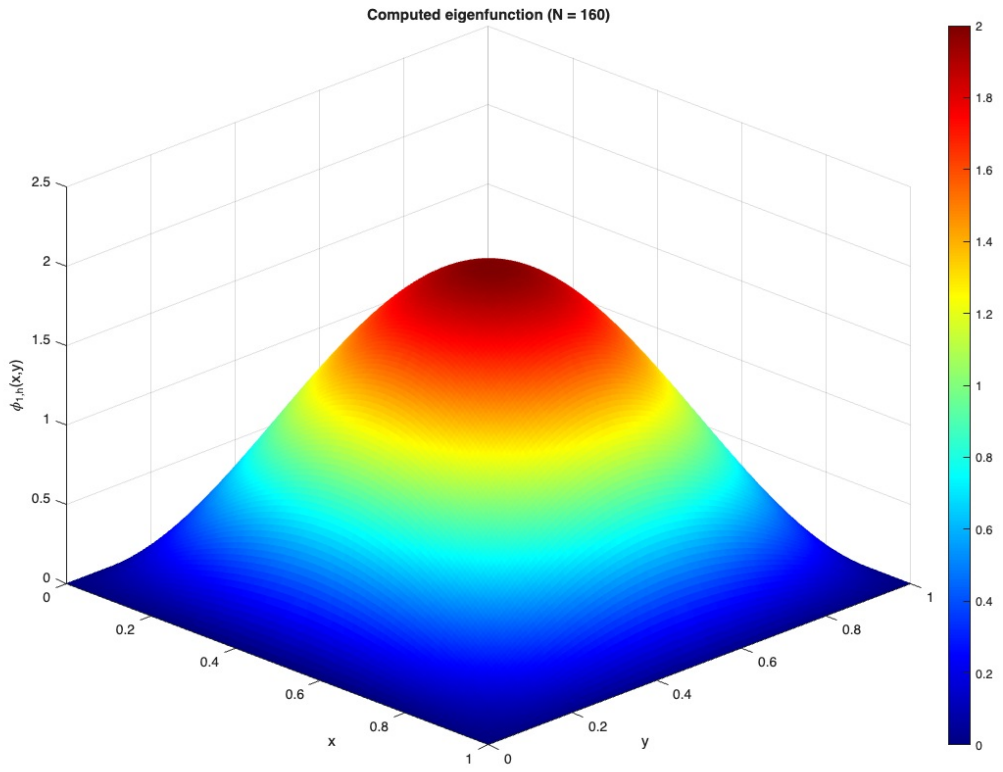


Figure 9: Computed principal eigenfunction $\phi_{1,h}(x, y)$ on the unit square for $N = 160$ (fine mesh)

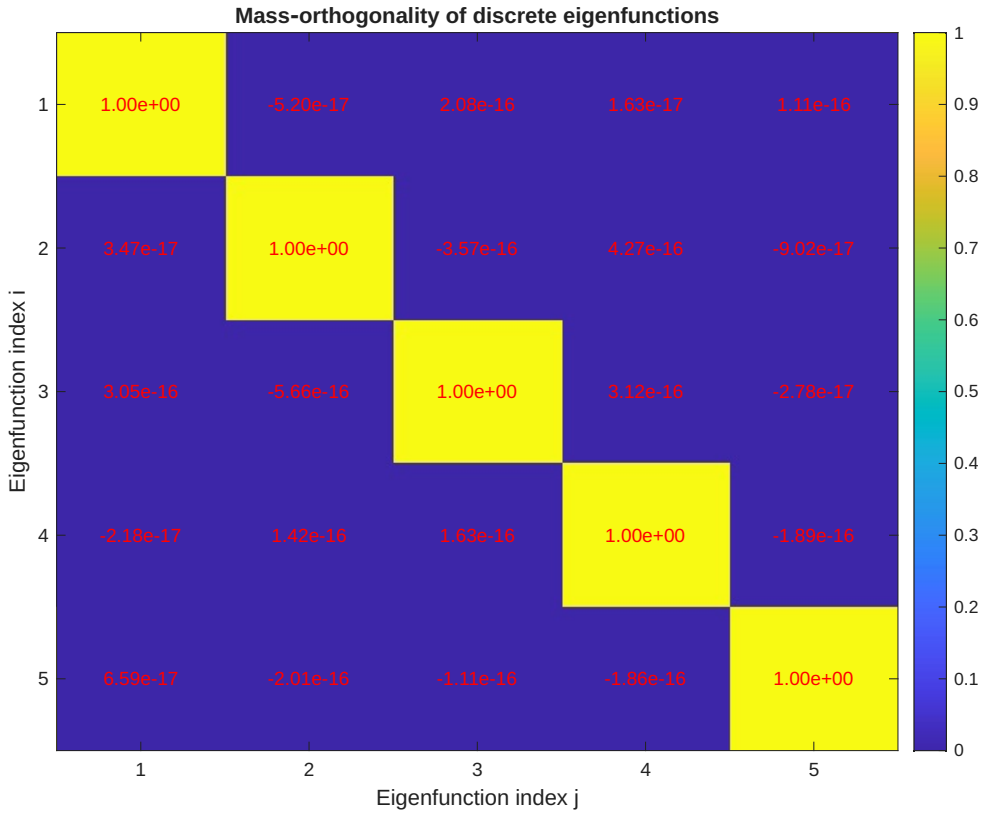


Figure 10: Matrix of normalised inner products $G_{ij} = \frac{\boldsymbol{\phi}_i^\mathsf{T} M_h \boldsymbol{\phi}_j}{\sqrt{(\boldsymbol{\phi}_i^\mathsf{T} M_h \boldsymbol{\phi}_i)(\boldsymbol{\phi}_j^\mathsf{T} M_h \boldsymbol{\phi}_j)}}$

In Figure 10 we show the Gram matrix of the first five discrete eigenfunctions $\boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_5$ with respect to the finite element mass matrix M_h:

$$A_h \boldsymbol{\phi}_i = \lambda_{i,h} M_h \boldsymbol{\phi}_i, \qquad i = 1, \ldots, 5.$$

The matrix G is defined by

$$G_{ij} = \boldsymbol{\phi}_i^{\mathsf{T}} M_h \boldsymbol{\phi}_j,$$

and then normalised so that $G_{ii} = 1$. Hence G_{ij} represents the discrete L^2 inner product between the i-th and j-th eigenfunctions.

The theoretical properties of the continuous Laplacian guarantee that eigenfunctions corresponding to distinct eigenvalues are orthogonal in $L^2(\Omega)$:

$$\int_\Omega \phi_i(x)\, \phi_j(x)\, dx = 0 \quad \text{for } i \neq j.$$

The finite element discretisation preserves this property up to machine precision, provided the same mass matrix is used to define the discrete inner product. The plot shows that the off-diagonal entries of G are of the order 10^{-15} to 10^{-14}, i.e., effectively zero. The diagonal entries are exactly 1 by construction. The orthogonality is a fundamental consequence of the self-adjointness of the Laplacian. It guarantees that the computed modes can be used as a basis for representing arbitrary functions.

Remark 6.1. *The numerical results confirm the theoretical expectations:*

1. ***One-sided error**: All computed eigenvalues $\lambda_{1,h}$ are greater than the exact λ_1, as expected from the variational characterization.*
2. ***Quadratic convergence**: The error decreases by a factor of 4 each time h is halved.*

6.5 Application: Sand on a square drumhead

Figure 11 shows contour plots of the first four finite element eigenfunctions. The exact eigenfunctions are

$$\phi_{m,n}(x, y) = \sin(m\pi x) \sin(n\pi y), \qquad \lambda_{m,n} = \pi^2(m^2 + n^2), \quad m, n = 1, 2, 3, \ldots$$

The first four distinct eigenvalues (ignoring multiplicities) are:

$$\begin{aligned} \lambda_{1,1} &= 2\pi^2 \approx 19.7392, \\ \lambda_{1,2} = \lambda_{2,1} &= 5\pi^2 \approx 49.3480, \\ \lambda_{2,2} &= 8\pi^2 \approx 78.9568. \end{aligned}$$

Imagine sprinkling fine sand on a unit square drumhead vibrating at a single eigenfrequency. The sand settles only on the stationary **nodal lines**, giving a physical picture of each mode (Figure 11).

- **Mode 1** (fundamental): no interior nodes — sand collects only along the clamped boundary.

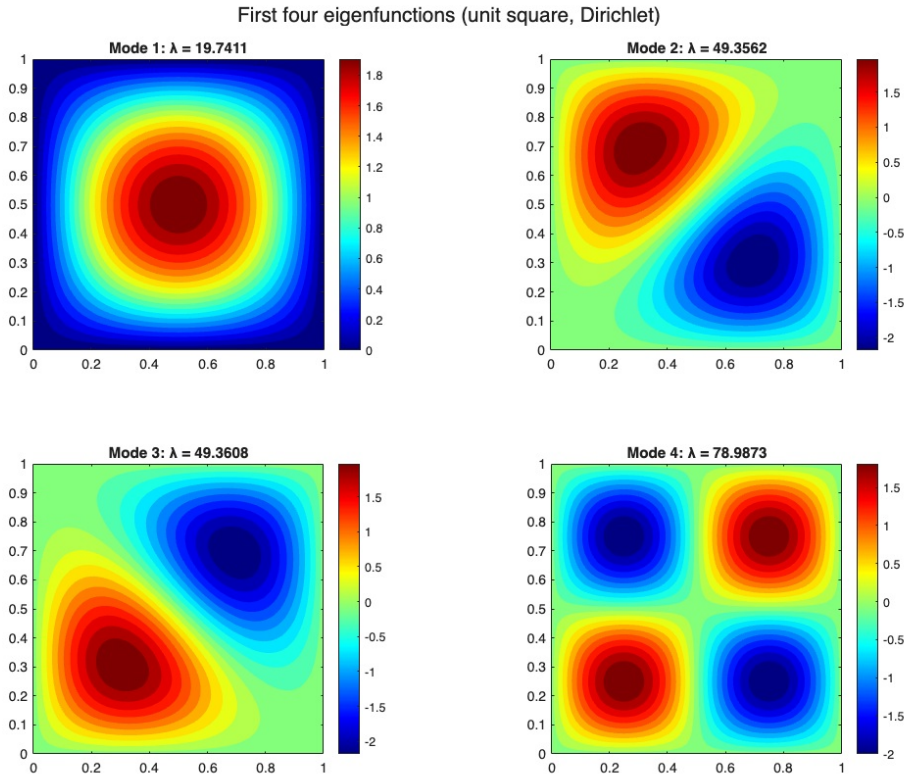


Figure 11: Contour plots of the first four finite element eigenfunctions.

- **Mode 2 & 3** (degenerate pair): one vertical or horizontal nodal line bisecting the square — sand forms a central stripe.
- **Mode 4**: a cross-shaped nodal set (vertical and horizontal lines) — sand traces a plus sign inside the boundary.

These nodal patterns match the analytic eigenfunctions $\sin(mx)\sin(ny)$ and confirm that the FEM solution captures the spectral geometry.

7 Conclusion and Future Work

In the previous sections, we used the compactness of the embedding $H_0^1 \hookrightarrow L^2$ to guarantee a discrete spectrum. We discretized the problem using P_1 finite elements, which transformed the infinite-dimensional eigenvalue problem into a generalized matrix eigenvalue problem $A_h\mathbf{u} = \lambda_h M_h\mathbf{u}$. Numerical experiments on the unit square confirmed the theoretical $O(h^2)$

convergence rate, with error ratios consistently approaching 4 as the mesh refined.

On convex polygons, elliptic regularity ensures that eigenfunctions belong to H^2. However, on non-convex domains, eigenfunctions exhibit corner singularities and belong only to $H^{1+\alpha}$ with $\alpha < 1$. Consequently, the convergence rate degrades to $O(h^{2\alpha})$. This motivates the use of adaptive finite element methods to recover optimal convergence by locally refining the mesh near singularities.

This report serves as a foundation for my summer research on adaptive finite element methods for spectral approximations in non-convex domains. Building on this work, we plan to study the spectral approximation of differential operators arising in quantum mechanics, with an initial focus on the Dirac operator and the phenomenon of spectral pollution. Unlike the Dirichlet Laplacian, the Dirac operator is non-coercive and indefinite, making it prone to spurious eigenvalues (spectral pollution) when standard Galerkin methods are used. We will begin with the stabilized Petrov–Galerkin framework [**almanasreh2026stability**, **hauck2026positivity**], to understand both the failure of standard Galerkin methods and the mechanisms by which stability can be restored.

Acknowledgements

We would like to thank APPM 6470 instructor, Prof. Nancy Rodriguez, for her guidance and lectures, which provided the PDE background necessary for this project.

Appendix

The following MATLAB script generates the mesh, assembles the stiffness and mass matrices, solves the generalized eigenvalue problem, and produces the convergence data in this report.

```
clear; clc; close all;

% Exact principal eigenvalue
lambda_exact = 2 * pi^2;

% Mesh sizes: h = 1/N
N_list = [10, 20, 40, 80, 160];
h_list = 1 ./ N_list;

dofs = zeros(length(N_list), 1);
lambda_h = zeros(length(N_list), 1);
errors = zeros(length(N_list), 1);
ratios = zeros(length(N_list), 1);

fprintf('N\t\th\t\tDeg. of Freedom\t\tlambda_h\t\tError\t\tRatio\n');

for k = 1:length(N_list)
    N = N_list(k);
    h = h_list(k);
    [A, M, interior_nodes] = assemble_matrices(N);

    A_int = A(interior_nodes, interior_nodes);
    M_int = M(interior_nodes, interior_nodes);

    opts.isreal = true;
    [~, lambda] = eigs(A_int, M_int, 1, 'smallestabs', opts);
    lambda_h(k) = lambda;

    errors(k) = abs(lambda_exact - lambda_h(k));
    dofs(k) = length(interior_nodes);

    if k == 1
        ratios(k) = NaN;
    else
        ratios(k) = errors(k-1) / errors(k);
    end

    fprintf('%d\t\t%.4f\t\t%d\t\t%.6f\t\t%.2e\t\t%.2f\n', ...
            N, h, dofs(k), lambda_h(k), errors(k), ratios(k));
end
row_names = arrayfun(@(x) sprintf('1/%d', x), N_list, 'UniformOutput',
false);
data = [h_list', dofs, lambda_h, errors, ratios];
ratio_str = cell(length(ratios), 1);
for i = 1:length(ratios)
    if isnan(ratios(i))
        ratio_str{i} = '---';
    else
        ratio_str{i} = sprintf('%.2f', ratios(i));
    end
```

```
end

headers = {'h', 'Deg. of Freedom', 'lambda_{1,h}', 'Error', 'Ratio'};
figure(2);
axis off;
set(gcf, 'Color', 'w', 'Units', 'normalized', 'Position', [0.2, 0.2, 0.6,
0.4]);
table_left = 0.1;
table_bottom = 0.1;
table_width = 0.8;
table_height = 0.8;
n_rows = length(N_list);
n_cols = length(headers);
cell_width = table_width / n_cols;
cell_height = table_height / (n_rows + 1);

add_centered_text = @(x, y, str, is_header) ...
    text(x, y, str, 'HorizontalAlignment', 'center', 'VerticalAlignment',
'middle', ...
         'FontSize', 11, 'FontWeight', iif(is_header, 'bold', 'normal'));
function out = iif(cond, if_true, if_false)
    if cond
        out = if_true;
    else
        out = if_false;
    end
end

for i = 0:n_rows
    for j = 0:n_cols-1
        x_left = table_left + j * cell_width;
        x_right = x_left + cell_width;
        y_top = table_bottom + (n_rows - i) * cell_height;
        y_bottom = y_top - cell_height;
        line([x_left, x_right, x_right, x_left, x_left], ...
             [y_top, y_top, y_bottom, y_bottom, y_top], ...
             'Color', 'k', 'LineWidth', 0.5);
        x_center = (x_left + x_right) / 2;
        y_center = (y_top + y_bottom) / 2;

        if i == 0   % header row
            txt = headers{j+1};
            is_header = true;
        else        % data row
            row = i;
            switch j
                case 0  % h
                    txt = sprintf('%.4f', data(row, 1));
                case 1  % def of freedoms
                    txt = sprintf('%d', data(row, 2));
```

```
                case 2  % lambda_h
                    txt = sprintf('%.4f', data(row, 3));
                case 3  % Error
                    txt = sprintf('%.2e', data(row, 4));
                case 4  % Ratio
                    txt = ratio_str{row};
            end
            is_header = false;
        end
        add_centered_text(x_center, y_center, txt, is_header);
    end
end

title_text = 'Convergence of the principal eigenvalue lambda_{1,h} on the
unit square.';
text(table_left + table_width/2, table_bottom + table_height + 0.03, ...
     title_text, 'HorizontalAlignment', 'center', 'FontSize', 12,
'FontWeight', 'bold');

function [A, M, interior_nodes] = assemble_matrices(N)
    x = linspace(0, 1, N+1);
    y = linspace(0, 1, N+1);
    [X, Y] = meshgrid(x, y);
    nodes = [X(:), Y(:)];
    n_nodes = (N+1)^2;

    n_elements = 2 * N * N;
    elements = zeros(n_elements, 3);
    idx = 0;
    for j = 1:N
        for i = 1:N
            bl = (j-1)*(N+1) + i;
            br = (j-1)*(N+1) + i+1;
            tl = j*(N+1) + i;
            tr = j*(N+1) + i+1;
            idx = idx + 1;
            elements(idx, :) = [bl, br, tr];
            idx = idx + 1;
            elements(idx, :) = [bl, tr, tl];
        end
    end

    A = sparse(n_nodes, n_nodes);
    M = sparse(n_nodes, n_nodes);

    for e = 1:n_elements
        nodes_e = elements(e, :);
        coord = nodes(nodes_e, :);
        x1 = coord(1,1); y1 = coord(1,2);
        x2 = coord(2,1); y2 = coord(2,2);
```

```
        x3 = coord(3,1); y3 = coord(3,2);
        area = 0.5 * abs((x2-x1)*(y3-y1) - (x3-x1)*(y2-y1));

        b = zeros(3,1);
        c = zeros(3,1);
        b(1) = (y2 - y3) / (2*area);
        c(1) = (x3 - x2) / (2*area);
        b(2) = (y3 - y1) / (2*area);
        c(2) = (x1 - x3) / (2*area);
        b(3) = (y1 - y2) / (2*area);
        c(3) = (x2 - x1) / (2*area);

        K_local = area * (b*b' + c*c');
        M_local = area/12 * [2, 1, 1; 1, 2, 1; 1, 1, 2];

        for i = 1:3
            for j = 1:3
                A(nodes_e(i), nodes_e(j)) = A(nodes_e(i), nodes_e(j)) +
K_local(i,j);
                M(nodes_e(i), nodes_e(j)) = M(nodes_e(i), nodes_e(j)) +
M_local(i,j);
            end
        end
    end

    tol = 1e-12;
    on_boundary = (abs(nodes(:,1)) < tol) | (abs(nodes(:,1)-1) < tol) | ...
                  (abs(nodes(:,2)) < tol) | (abs(nodes(:,2)-1) < tol);
    interior_nodes = ~on_boundary;
end


lambda_exact = 2 * pi^2;
N_list = [10, 20, 40, 80, 160];
h_list = 1 ./ N_list;

lambda_h = zeros(length(N_list), 1);
errors = zeros(length(N_list), 1);

for k = 1:length(N_list)
    N = N_list(k);
    h = h_list(k);


    [A, M, interior_nodes] = assemble_matrices(N);
    A_int = A(interior_nodes, interior_nodes);
    M_int = M(interior_nodes, interior_nodes);

    opts.isreal = true;
    [V, lambda] = eigs(A_int, M_int, 1, 'smallestabs', opts);
```

```
    lambda_h(k) = lambda;
    errors(k) = abs(lambda_exact - lambda_h(k));

    n_nodes_total = (N+1)^2;
    phi_full = zeros(n_nodes_total, 1);
    phi_full(interior_nodes) = V(:,1);

    x = linspace(0, 1, N+1);
    y = linspace(0, 1, N+1);
    [Xg, Yg] = meshgrid(x, y);
    nodes = [Xg(:), Yg(:)];

    tol = 1e-12;
    on_boundary = (abs(nodes(:,1)) < tol) | (abs(nodes(:,1)-1) < tol) | ...
                  (abs(nodes(:,2)) < tol) | (abs(nodes(:,2)-1) < tol);

    % Exact eigenfunction
    phi_exact = sin(pi * nodes(:,1)) .* sin(pi * nodes(:,2));
    phi_exact(on_boundary) = 0;
    e_vec = phi_exact - phi_full;

end


k_fine = length(N_list);
N_fine = N_list(k_fine);
[A_fine, M_fine, interior_fine] = assemble_matrices(N_fine);
A_int_fine = A_fine(interior_fine, interior_fine);
M_int_fine = M_fine(interior_fine, interior_fine);
[V_fine, ~] = eigs(A_int_fine, M_int_fine, 1, 'smallestabs');
phi_full_fine = zeros((N_fine+1)^2, 1);
phi_full_fine(interior_fine) = V_fine(:,1);
X_fine = linspace(0, 1, N_fine+1);
Y_fine = linspace(0, 1, N_fine+1);
Phi_grid = reshape(phi_full_fine, N_fine+1, N_fine+1);

figure(3);
surf(X_fine, Y_fine, Phi_grid, 'EdgeColor', 'none');
colormap(jet);
colorbar;
xlabel('x'); ylabel('y'); zlabel('\phi_{1,h}(x,y)');
title(sprintf('Computed eigenfunction (N = %d)', N_fine));
view(45, 30);
[V4, D4] = eigs(A_int_fine, M_int_fine, 4, 'smallestabs');
lambda4 = diag(D4);
[~, idx] = sort(lambda4);
V4 = V4(:, idx);
figure;
for i = 1:4
    phi_full = zeros((N_fine+1)^2, 1);
```

```
    phi_full(interior_fine) = V4(:, i);
    Phi_grid = reshape(phi_full, N_fine+1, N_fine+1);

    subplot(2,2,i);
    contourf(X_fine, Y_fine, Phi_grid, 20, 'LineColor', 'none');
    colormap(jet);
    colorbar;
    title(sprintf('Mode %d: lambda = %.4f', i, lambda4(i)));
    axis equal tight;
end
sgtitle('First four eigenfunctions (unit square, Dirichlet)');
figure;
subplot(1,2,1);
spy(A, 5);
title('Stiffness matrix A');
subplot(1,2,2);
spy(M, 5);
title('Mass matrix M');
N_fine = 80;
[A_fine, M_fine, interior_fine] = assemble_matrices(N_fine);
A_int = A_fine(interior_fine, interior_fine);
M_int = M_fine(interior_fine, interior_fine);
[V, D] = eigs(A_int, M_int, 5, 'smallestabs');
[D_sort, idx] = sort(diag(D));
V = V(:, idx);
G = V' * M_int * V;
G = G ./ sqrt(diag(G) * diag(G)');

figure;
imagesc(G);
colorbar;
colormap(parula);
xticks(1:5); yticks(1:5);
xlabel('Eigenfunction index j');
ylabel('Eigenfunction index i');
title('Mass orthogonality of discrete eigenfunctions');
for i = 1:5
    for j = 1:5
        text(j, i, sprintf('%.2e', G(i,j)), ...
            'HorizontalAlignment', 'center', 'Color', 'r');
    end
end
N = 20;
h = 1/N;

% Node coordinates
x = linspace(0, 1, N+1);
y = linspace(0, 1, N+1);
[X, Y] = meshgrid(x, y);
nodes = [X(:), Y(:)];
```

```
n_nodes = size(nodes, 1);

elements = [];
for j = 1:N
    for i = 1:N
        bl = (j-1)*(N+1) + i;
        br = bl + 1;
        tl = j*(N+1) + i;
        tr = tl + 1;
        % Triangle 1
        elements = [elements; bl, br, tr];
        % Triangle 2
        elements = [elements; bl, tr, tl];
    end
end
n_elements = size(elements, 1);

figure;
triplot(elements, nodes(:,1), nodes(:,2), 'k-');
axis equal;
title('Triangle mesh (P1 elements)');
xlabel('x'); ylabel('y');
```